AgentWatch: A Flight Recorder and Report Card for AI Agents

Mahesh Thakur, IIT Patna (Mathematics and Computing)

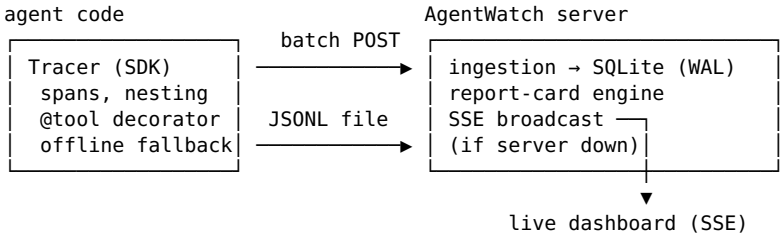
October 2026

Overview

AgentWatch is an observability system for AI agents. When an agent runs a task, AgentWatch records every step - tool calls, model calls, tokens, cost, latency - replays the run as a live waterfall on a dashboard, and grades it with an explainable report card. It answers the question every team deploying agents eventually asks: **what did the agent actually do, and was that good?**

The system has four parts: a dependency-free tracing SDK, a FastAPI ingestion and query server with SQLite storage, a deterministic report-card engine, and a live dashboard fed by Server-Sent Events. A demo agent with three scripted scenarios proves the full pipeline end to end without any paid API.

Architecture



Design rule: observability must never break the observed system. The SDK uses only the Python standard library, flushes on a background thread, and falls back to a local JSONL file when the server is unreachable. No trace is ever silently lost.

The report card

Grades come from deterministic rules, not from a model's mood. Five checks, each with a named severity and point cost:

Check	What it catches	Severity
Repeated-action loop	same tool + identical input ≥3× in a row	critical (−40)
Loop dominated the run	loop calls ≥ 50% of all tool calls	major (−20)
High failure rate	>30% of steps in error state	major (−20)
Over cost budget	trace cost above the configured budget	major (−20)
Activity, no answer	≥5 tool calls but no final answer	major (−20)
One step dominated	a single span took >60% of the run	minor (−10)

Every card names the behaviour that cost points, in plain language, so a developer can act on it. The demo suite's stuck-in-a-loop run grades D (40/100) with "Repeated-action loop" and "Loop dominated the run"; the clean and failure-recovery runs grade A. These outcomes are asserted in the automated test suite.

Measured performance

All figures are measured by `agentwatch/benchmark.py` on the development machine (raw record in `docs/benchmarks.json`), not estimated:

- **~3.9 µs overhead per recorded step** (0.15 µs untraced vs 4.01 µs traced function call)
- **~1,400 spans/second end-to-end ingestion**, including SDK batching, HTTP transport, SQLite writes and running the report card on every trace
- **2.4 ms** for fleet-wide statistics over 200 stored traces

Testing

20 automated tests cover the SDK (nesting, I/O capture, error capture, offline fallback, field clipping), storage (round-trip, aggregation, percentiles, eval persistence), the report card (every rule and grade boundary), the HTTP API (ingest, list, detail, re-evaluate, 404s, malformed input), and one end-to-end test that runs the real demo agent through the real API and asserts the loop is caught and graded down. CI runs the suite on every push.

Honest limitations

- The demo planner is scripted so the project runs with zero API keys; instrumenting a real LLM-driven agent is the same two lines of SDK.
- Cost and token figures are what the instrumented agent reports; AgentWatch never invents usage.
- SQLite targets the single-operator case; the storage layer is one class and swappable for Postgres without touching the API.
- An optional LLM judge could add qualitative notes, but the grade will always remain rule-based and explainable.